

CRUISER Product Roadmap

Document Info

- Project: `CRUISER`
- Repository: `ferhanburak/carfollow`
- Last updated: `2026-07-13`
- Scope: product status, current architecture, backend direction, roadmap, technical debt

1. Executive Summary

`CRUISER` is no longer a static concept demo. It has evolved into a working prototype with:

- modular React frontend structure
- Firebase-backed persistence and realtime foundations
- vehicle passport and maintenance tracking
- social graph and direct messaging features
- convoy trust rules and gated participation
- separate `Grid Map` and `Live Map` experiences
- convoy lifecycle simulation on top of Google Maps

The project is currently at a strong `MVP+` stage.

What is already strong:

- product identity is clear
- differentiator is visible: `social + convoy + trust + live map + vehicle history`
- core flows are usable

What still needs maturity:

- data model normalization
- centralized backend business rules
- social/profile interaction polish
- admin/moderation and auditability
- production-level test and security hardening

2. Current Product Status

2.1 Completed

Area	Status	Notes
Authentication UI	Complete	Login, signup, test profiles
Modular frontend structure	Complete	Screens/components separated into maintainable files
Firebase integration baseline	Complete	Firestore + RTDB + rules + app config
Grid Map node management	Complete	Spot, wash, meet creation flows
Live Map screen	Complete	Separate page with Google Maps
Fuel log	Complete	Form + history + insights

Service log	Complete	Manual maintenance logging	
Vehicle passport	Complete	Part health, history, ownership-value direction	
Individual leaderboard	Complete	Monthly KM ranking	
Clan leaderboard	Complete	Collective KM ranking	
Friend requests	Complete	Add, incoming, outgoing, withdraw	
Social profile drawer	Complete	Public driver view baseline	
Direct messages	Complete	Thread list, send, typing, presence	
Clan invites	Complete	Basic create/accept/decline/revoke	
Convoy trust rules	Complete	Score/harmony/alert restrictions	
Convoy request moderation	Complete	Join request, host approve/decline	
Convoy lifecycle statuses	Complete	`planning`, `delayed`, `rolling`, `completed`	
Attendee trip statuses	Complete	`ready`, `enroute`, `arrived`, `cancelled`	
Live convoy timeline	Complete	ETA, progress, driven/remaining distance	
Ghost convoy vehicles	Complete	Moving mini markers on route	
Live marker info bubbles	Complete	Driver status + quick actions	
Bubble social actions	Complete	Open profile, add friend, send DM	

2.2 In Progress / Partially Solved

Area	Status	Notes	
---	---	---	
Public profile opening from anywhere	Partial	Exists through social view, not yet fully centralized	
Social UX flow consistency	Partial	Some actions redirect rather than open unified overlays	
Convoy host tooling	Partial	Core gating exists, moderation dashboard still limited	
Firebase data ownership boundaries	Partial	Works, but domain separation should be clearer	

2.3 Not Yet Implemented

Area	Priority	Notes	
---	---	---	
Vehicle passport export / resale report	High	Major product differentiator	
Full clan management	High	Roles, members, promote/remove flows	
Convoy moderation panel	High	Kick, ban, access audit, host logs	
Notification center	Medium	Friend, clan, convoy, DM events	
Public driver profile overlay system	Medium	Central reusable profile surface	
Media storage pipeline	Medium	Spot photos should move to real storage	
Admin / moderation tools	Medium	Needed before scale	
Analytics / event logging	Medium	Needed for product decisions	

3. Strengths and Weaknesses

3.1 Strengths

- The product has a clear identity.

- The best differentiator is already working: convoy trust and live visualized group driving.
- Service history and vehicle health give the app a real utility layer beyond social novelty.
- Live Map now feels product-like rather than mock-like.
- Social graph, chat, and live convoy are connected instead of isolated features.
- The frontend architecture is much more maintainable than the original single-file state.

3.2 Weaknesses

- Some domain actions still live too close to the client.
- A few large files are growing and should eventually be split again.
- User, vehicle, and convoy ownership rules are not yet fully normalized for long-term scale.
- Social actions are functional, but not yet deeply polished.
- There is not yet a full backend audit trail for important trust/moderation actions.

3.3 Technical Debt

- `src/components/MapCard.jsx`
Large and feature-rich, but approaching split threshold.
- `src/screens/StatsScreen.jsx`
Handles many social surfaces and should later be decomposed.
- `src/App.jsx`
Still centralizes a lot of orchestration.
- Domain logic is partly in hooks and partly in repository helpers.
- Client-side business rules should gradually move to secure backend functions.

4. Recommended Backend Architecture

4.1 Short Answer

The best backend for the current phase is:

- `Firebase Auth` for identity
- `Firestore` for persistent product data
- `Realtime Database` for low-latency live data
- `Cloud Functions` for business rules and secure mutations

This is the right balance between speed, cost, maintainability, and feature fit.

4.2 Backend Responsibilities

The backend should be responsible for:

- authenticating users
- validating write operations
- enforcing convoy trust rules
- controlling friend and clan mutations
- logging important events
- updating reputation safely

- protecting data ownership boundaries
- producing export/report documents later

The frontend should not be the final authority for:

- convoy access approval
- score/reputation changes
- clan membership changes
- ownership transfers
- moderation decisions

4.3 Recommended Layering

Layer 1: Identity

- `Firebase Auth`
- UI may still use plate-centric experience
- internal identity must be `userId`

Layer 2: Domain Logic

- `Cloud Functions`
- Secure mutation endpoints for:
 - friend request create/accept/reject
 - convoy join request
 - convoy host approval
 - clan invite create/accept/revoke
 - score and trust updates
 - ownership/passport transfer workflows

Layer 3: Persistent Data

- `Firestore`
- Good for:
 - user profiles
 - vehicles
 - convoy metadata
 - clan metadata
 - service logs
 - fuel logs
 - passport summaries

Layer 4: Realtime Data

- `Realtime Database`
- Good for:
 - direct messages
 - typing state
 - presence
 - live telemetry
 - live convoy participant movement

5. Recommended Data Model

Core Identity Model

Do not rely on `plate` as the only identity.

Use:

- `userId` = system identity
- `vehicleId` = vehicle identity
- `plate` = public-facing social identity

Why:

- users may change vehicles
- vehicles may change owners
- passport history should survive ownership transfer
- future multi-vehicle support becomes possible

Suggested Firestore Collections

```
```text
/users/{userId}
/vehicles/{vehicleId}
/vehiclePassports/{vehicleId}
/vehicles/{vehicleId}/serviceLogs/{logId}
/vehicles/{vehicleId}/fuelLogs/{logId}
/clans/{clanId}
/clans/{clanId}/members/{userId}
/convoys/{convoyId}
/convoys/{convoyId}/members/{userId}
/mapNodes/{nodeId}
/friendships/{friendshipId}
/notifications/{userId}/items/{notificationId}
```
```

Suggested RTDB Paths

```
```text
/presence/{userId}
/typing/{conversationId}/{userId}
/dmThreads/{conversationId}
/liveTelemetry/{convoyId}/{userId}
/liveConvoyState/{convoyId}/{userId}
```
```

Domain Separation

User

- full name
- avatar
- social settings
- reputation snapshot
- primary vehicle reference

Vehicle

- plate
- model
- tuning stage
- horsepower

- odometer
- owner user reference

Vehicle Passport

- badges
- maintenance summary
- resale summary
- health summary

Convoy

- host
- visibility
- trust thresholds
- route metadata
- lifecycle status
- member list

Social Graph

- friendships
- requests
- conversations
- presence

6. Product Roadmap

6.1 Short-Term Plan

Target: stabilize and polish the current MVP+

Product

- centralize `public profile` opening from all surfaces
- improve Live Map bubble actions UX
- smooth DM entry flow from profile and map
- clean up social screen grouping and labels

Technical

- split `MapCard.jsx` into smaller map layers
- split `StatsScreen.jsx` into social modules
- add more tests for convoy and social logic
- move sensitive mutations toward functions/repository boundary

Deliverables

- unified profile overlay
- cleaner social navigation
- stronger map interaction polish
- roadmap-aligned documentation

6.2 Mid-Term Plan

Target: convert prototype into a more trustworthy platform

Product

- convoy host moderation dashboard
- clan management improvements
- notification center
- vehicle passport resale/export flow
- stronger ownership history records

Technical

- introduce Cloud Functions for secure writes
- normalize Firestore collections
- add audit logs for:
 - convoy moderation
 - trust score changes
 - service updates
 - ownership transfer actions

Deliverables

- secure convoy workflow
- reliable service/passport domain
- clearer moderation features

6.3 Long-Term Plan

Target: scale into a real platform

Product

- ownership transfer package for selling a vehicle
- marketplace-adjacent verified history concept
- richer convoy discovery and route intelligence
- admin dashboard
- moderation and trust analytics

Technical

- evaluate hybrid backend:
 - Firestore + RTDB remain
 - PostgreSQL added for deep reporting and relational workflows
- analytics pipeline
- background jobs
- document export service

Deliverables

- scalable backend model
- business-ready data integrity
- stronger long-term product value

7. What Should Be Added

- centralized public profile system
- stronger convoy host controls

- resale-ready vehicle passport export
- notification center
- audit logs
- stronger image/media storage flow
- domain-level backend functions

8. What Should Be Refactored

- `src/components/MapCard.jsx`
- `src/screens/StatsScreen.jsx`
- `src/App.jsx`
- state orchestration boundaries between hooks and repositories
- shared profile opening logic

9. What Should Not Be Rushed Yet

- full backend migration away from Firebase
- premature PostgreSQL adoption
- overly complex microservice split
- advanced analytics before core flows are stable

The current product still benefits more from:

- rule hardening
- data normalization
- product flow polish

than from heavy infrastructure expansion.

10. Recommended Next 5 Tasks

1. Create a centralized `public profile overlay` service usable from map, social, leaderboard, and DM.
2. Move friend request and convoy join approval logic to secure backend functions.
3. Build a host-focused convoy moderation panel with invite/request/member states.
4. Design and implement `Vehicle Passport Export` for resale/ownership handoff.
5. Split large UI files and add focused tests around convoy trust, social actions, and profile actions.

11. Final Assessment

The project is in a very promising place.

It already has:

- a real product personality
- a believable core loop

- a differentiating value proposition
- enough technical structure to continue safely

The immediate mission is no longer "build the app".

The mission now is:

- normalize
- secure
- polish
- document
- deepen the strongest features

That is a strong position to be in.